Claude Code Telemetry Analytics & Insights

Monitoring Usage, Optimizing Costs, & Predicting Token Demand

Project Overview

- **Objective:** Full-stack operational telemetry and predictive modeling for Claude Code usage.
- **Tech Stack:** Python, SQLite, Streamlit, Scikit-learn, Plotly.
- **Key Outcome:** Unified visibility into tool execution, engineering domain costs, and developer prompt anomalies.

End-to-End System Architecture

Automated Data Pipeline & Analytics Core

1. Ingestion

- Parses directory records and interaction telemetry logs.
- Stores structured data in indexed SQLite warehouse.

2. Analytics & ML

- SQL aggregations for cost distributions & tool usage.
- Dual-ML engine for forecasting & session anomaly detection.

3. UI Dashboard

- Streamlit interface with interactive visual charts.
- Real-time telemetry metric breakdowns.

Usage & Cost Analytics

Granular Cost & Tool Efficiency Analysis

Token Cost by Practice Area

- Categorizes expenditure across engineering practices (ML, Frontend, Backend, Data, Platform).
- Pinpoints high-consumption development teams.

Tool Decision Breakdown

- Tracks total execution tool approvals versus rejections.
- Monitors performance across Read, Edit, Bash, and Grep tools.

Machine Learning Engines

Predictive Cost Modeling & Anomaly Detection

Daily Cost Forecasting

- **Model:** Linear Regression
- Fits historical daily token volume to predict upcoming developer expenditure trends.

Session Anomaly Detection

- **Model:** IsolationForest
- Evaluates multi-dimensional session vectors to identify unusual cost spikes and prompt behaviors.

Automated Orchestration

One-Command Master Script Execution (`run.py`)

Key Execution Steps

- Validates database status and ingests raw logs if necessary.
- Runs validation tests on analytics queries and ML models.
- Launches interactive Streamlit interface automatically.
- Provides zero-configuration execution for fresh environments.